Blockchain & Secure Multi-Party Computation

Master 2 — Computer Science

Session 3 of 12 — Cryptography for Blockchain I

Where We Are

Blockchain's core guarantees — immutability, authenticity, and trustless verification — are entirely built on cryptographic primitives. These primitives were studied in depth in the IoT Security module (Sessions 2–5 of that course). This session and the next review and connect them specifically to their blockchain roles, adding only what is new to this context. This session covers hash functions, digital signatures, and encryption in their blockchain applications. Session 4 covers consensus mechanisms, which are the other major cryptographic component specific to blockchain.

1 Hash Functions in Blockchain

Definition — Cryptographic Hash Function — Blockchain Context

A cryptographic hash function $H : \{0,1\}^* \to \{0,1\}^n$ maps any input (a transaction, a block, a document) to a fixed-size output called the hash or "digest." In blockchain, H must satisfy: **determinism** (same input always produces same output), **pre-image resistance** (given $h = H(x)$, cannot efficiently find x), **second pre-image resistance** (given x, cannot efficiently find $x' \neq x$ with $H(x') = H(x)$), **collision resistance** (cannot efficiently find any pair x, x' with $H(x) = H(x')$), and the **avalanche effect** (a single-bit change in input completely changes the output). Bitcoin uses SHA-256 (applied twice: SHA-256(SHA-256(data))); Ethereum uses Keccak-256.

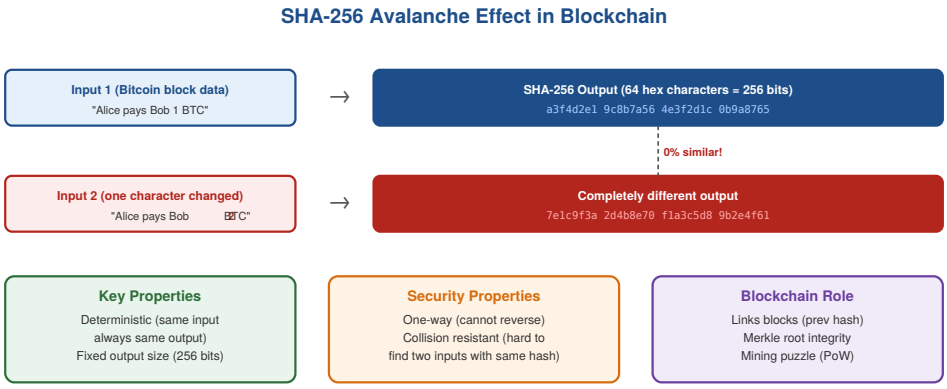


Figure 1: The avalanche effect: changing "1 BTC" to "2 BTC" — a single character — produces a completely unrecognizable output. This property ensures that no one can make a small, undetected change to a transaction or block.

1.1 How Hash Functions Secure the Blockchain

Hash functions serve three distinct roles in a blockchain, each exploiting a different property:

Role 1 — Block linking. Each block stores H(previous block) in its header. Any modification to a past block changes its hash, breaking the link to the next block and every block after it. The entire modified chain becomes invalid because it diverges from the chain held by the rest of the network.

Role 2 — Transaction integrity (Merkle tree). All transactions in a block are organized into a Merkle tree (Session 1), where the Merkle Root is a single hash summarizing all of them. Any change to any transaction changes the Merkle Root, which is stored in the block header, which changes the block's hash.

Role 3 — Proof of Work mining puzzle. In Proof of Work (Session 4), miners must find a nonce value such that H(block header$\|$nonce) begins with a required number of zero bits. Since hash outputs are pseudo-random and unpredictable, the only strategy is to try billions of nonce values until one works. This computational difficulty is what makes rewriting history expensive.

Example — A Bitcoin block hash in practice

A valid Bitcoin block hash looks like:

`00000000000000000025695bdb1c8fe1f04f87b9d69ee29f2c53b9ed5a`

The long sequence of leading zeros is not a coincidence — the mining algorithm specifically required finding a nonce that produces a hash with this many leading zero bits. The current difficulty requires approximately 18 leading zero bits in the hash, meaning a miner tries on average $2^{18} \approx 260{,}000$ nonces before finding a valid one per target difficulty adjustment.

? Think about it

If SHA-256 is a public algorithm that everyone knows, what prevents an attacker from pre-computing many valid block hashes and using them to rewrite history?

✓ Answer

The pre-image resistance of SHA-256: there is no shortcut. To produce a hash with k leading zeros, an attacker must try 2^k inputs on average. To rewrite a past block, they must redo this work for that block AND every block after it, faster than the rest of the network is adding new blocks. At Bitcoin's scale (hundreds of exahashes per second collectively), this is computationally and economically infeasible.

2 Digital Signatures in Blockchain

Definition — Digital Signature — Blockchain Context

In blockchain, a digital signature on a transaction proves two things: (1) **authentication** — the transaction was authorized by the holder of the private key corresponding to the sending address (no one else could have produced this signature); (2) **non-repudiation** — the sender cannot later deny having authorized the transaction, because only they hold the private key. Formally, for signing key sk and verification key pk: $\sigma = \text{Sign}(sk, H(tx))$ and $\text{Verify}(pk, H(tx), \sigma) = 1$ if and only if σ was produced with the sk corresponding to pk.

2.1 ECDSA: The Signature Scheme Bitcoin Uses

Bitcoin uses the Elliptic Curve Digital Signature Algorithm (ECDSA) with the secp256k1 curve. The private key sk is a 256-bit random integer; the corresponding public key is a point on the elliptic curve $Q = sk \cdot G$, where G is the curve's base point. The blockchain address is then derived

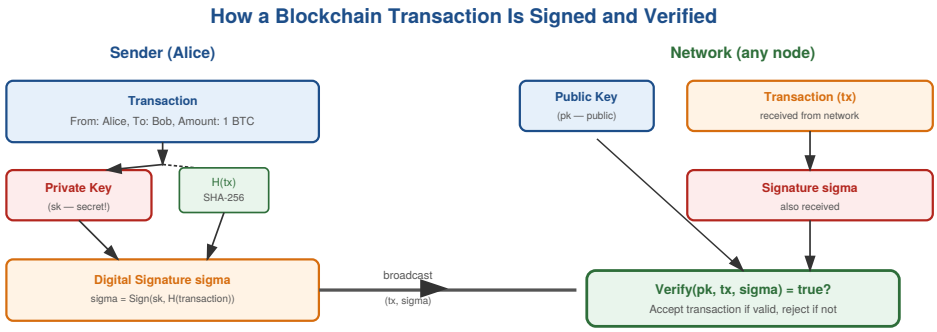


Figure 2: Transaction signing and verification. The sender signs the transaction's hash with their private key; any network node can verify the signature using the sender's public key, without ever learning the private key.

as:

$$\text{address} = \text{Base58Check}(\text{RIPEMD160}(\text{SHA256}(Q)))$$

The two-step hash (SHA256 followed by RIPEMD160) produces a 160-bit (20-byte) address, which is shorter and therefore easier to share. The Base58Check encoding adds a checksum to detect typos.

Example — Why Ethereum uses Keccak-256 instead of SHA-256

Ethereum was designed after Bitcoin and deliberately chose Keccak-256 (the original version of SHA-3 before it was modified in the NIST standardization process) for several reasons: it is faster to compute in the Ethereum Virtual Machine, it provides different security assumptions from SHA-256 (reducing correlated vulnerabilities), and its sponge construction allows flexible output lengths useful for Ethereum's internal operations.

2.2 Schnorr Signatures and Taproot

Bitcoin activated Taproot in November 2021, introducing Schnorr signatures as an alternative to ECDSA. Recall from the IoT Security module that Schnorr signatures have a clean algebraic structure:

$$\text{Sign: } (R, s) \text{ where } R = k \cdot G, \quad c = H(R \| pk \| m), \quad s = k + c \cdot sk$$

$$\text{Verify: } s \cdot G = R + H(R \| pk \| m) \cdot pk$$

In blockchain, Schnorr's key advantage is **linearity**: multiple parties' partial signatures can be combined into a single signature indistinguishable from a regular single-signer signature (key aggregation/MuSig2, studied in the IoT module). For Bitcoin, this means a multi-signature wallet (requiring, say, 3 out of 5 signers) produces exactly the same-size transaction as a single-signer wallet, saving blockchain space and improving privacy by hiding multi-party structure.

? Think about it

Why is hiding the multi-party structure of a Bitcoin transaction useful for privacy?

✓ Answer

Without Schnorr/Taproot, a 3-of-5 multisig transaction is visibly different on the blockchain from a regular transaction — anyone can see it requires multiple signers and count how many. This reveals information about the wallet's security model and can help attackers target

high-value multi-party wallets. With Schnorr key aggregation, the combined signature looks identical to a single-person transaction, providing privacy equivalent to that of the simplest possible wallet type.

3 Asymmetric Encryption in Blockchain

Asymmetric encryption (RSA, ECDH) was covered in depth in the IoT Security module (Session 2 of that course). In blockchain, asymmetric encryption appears in two specific contexts:

Context 1 — Key exchange for encrypted messaging. Blockchain protocols that support peer-to-peer encrypted messaging (such as Ethereum's Whisper/Waku protocol) use ECDH (Elliptic Curve Diffie-Hellman) to establish a shared symmetric key between two parties, then encrypt messages with AES. The blockchain itself carries the encrypted messages; only the intended recipient (holding the matching private key) can decrypt them.

Context 2 — Off-chain data encryption. Applications built on blockchain often store actual data off-chain (in IPFS, a cloud database, or an encrypted file store) and put only the hash (and optionally an encrypted symmetric key) on-chain. The data is encrypted with a symmetric key under AES-GCM; the symmetric key itself is encrypted with the recipient's RSA or ECDH public key and stored on-chain.

Example — Medical data on IPFS + Ethereum

A patient's medical image is too large for the blockchain. The hospital encrypts it with AES-256-GCM (generating a random session key k), stores the encrypted image on IPFS (obtaining a content-addressed hash h), encrypts k with the patient's ECDH public key, and records the tuple $(h, \text{Enc}_{pk}(k))$ on Ethereum. The patient — and only the patient — can later retrieve the encrypted image using h and decrypt k using their private key to access the image.

? Think about it

Why not encrypt the data directly with the user's public key (RSA or ECDH), rather than using this hybrid scheme?

✓ Answer

Two reasons: efficiency and key agility. Asymmetric encryption is 100–1000 times slower than AES for large data. Encrypting a 50 MB medical image with RSA directly would take seconds on a server and would produce a ciphertext of the same size (not smaller). The hybrid scheme uses RSA/ECDH only for the tiny session key (typically 32 bytes) and AES for the bulk data. Additionally, the hybrid scheme allows changing the recipient's access without re-encrypting all the data — simply re-encrypt the session key under the new recipient's public key.

4 Homomorphic Encryption in Blockchain

Homomorphic encryption (HE) was studied in detail in the IoT Security module (Session 4). In blockchain, it serves a specific and important role: allowing computations on encrypted data without revealing the underlying values.

Example — Confidential smart contracts with HE

A smart contract on a public blockchain typically reveals all its data to the world (full transparency). For applications like sealed-bid auctions, payroll processing, or medical record

aggregation, this full transparency is unacceptable. Using Paillier homomorphic encryption (additively homomorphic — recall that $\text{Enc}(m_1) \cdot \text{Enc}(m_2) = \text{Enc}(m_1 + m_2)$), bidders can submit encrypted bids to a smart contract, which can sum or compare them without ever decrypting any individual bid. Only the final result (the winner) is revealed.

Example — Aztec Protocol: private transactions on Ethereum

The Aztec Protocol uses a combination of homomorphic commitments and zero-knowledge proofs (covered in Session 11) to enable private transactions on the Ethereum public blockchain. Transaction amounts are hidden behind cryptographic commitments (using the Pedersen commitment scheme, which is additively homomorphic), while zero-knowledge proofs convince the network that the arithmetic (inputs equal outputs, no negative amounts) is correct without revealing the actual numbers.

? Think about it

If homomorphic encryption can hide transaction data, why does not every blockchain use it by default?

✓ Answer

Computational cost. Fully homomorphic encryption (FHE) remains expensive: encrypting data is fast, but performing arbitrary computations on ciphertexts is currently several orders of magnitude slower than the same computation on plaintext. A smart contract that takes 10ms to execute on plaintext data might take seconds or minutes with FHE. Blockchain systems with limited transaction throughput cannot absorb this overhead for general use. Practical deployments use more targeted schemes (Paillier for aggregation, Pedersen commitments for confidential transactions) that provide specific useful homomorphic properties at reasonable cost.

5 Key Management and Wallets: The Human Side of Blockchain Cryptography

All the cryptographic guarantees of blockchain ultimately depend on the user's ability to securely manage their private keys. This is where most real-world losses occur.

Example — Hardware wallets: keeping the private key offline

A hardware wallet (such as a Ledger Nano or Trezor) is a small USB device that stores the user's private key in a secure element chip and performs all signature operations internally. The private key never leaves the device, even during a transaction signing operation. The host computer sends the transaction data to the hardware wallet; the wallet displays the details to the user for confirmation; the user presses a physical button; the wallet signs internally and sends back only the signature. Even if the host computer is fully compromised by malware, the attacker cannot extract the private key.

Example — Multi-signature wallets: distributing the risk

A 2-of-3 multi-signature Bitcoin wallet requires any 2 of 3 specified private keys to sign a transaction. This allows a setup where: Key 1 is on the user's phone (online but convenient), Key 2 is on a hardware wallet (offline, secure), Key 3 is in a bank vault (disaster recovery). No single key compromise loses the funds. Loss of one key is recoverable using the other two. This is widely used by cryptocurrency exchanges and high-net-worth individuals.

? Think about it

A user generates a Bitcoin private key on a computer, notes the public address, and sends 10,000 USD worth of Bitcoin to it, then formats the computer without having backed up the key. Is there any way to recover the funds?

✓ Answer

No. The private key exists only on the formatted drive. Without the key, no valid signature can be produced, and no transaction can spend those coins. Bitcoin Core (and all other wallet software) has no "forgot my key" recovery mechanism — this is by design, as any such mechanism would require a trusted central authority, which defeats the purpose. Professional data recovery services sometimes recover private keys from formatted drives if the data has not been overwritten, but success is not guaranteed. This illustrates a fundamental trade-off: true cryptographic self-sovereignty means total personal responsibility for key security.

Example — BIP-39 seed phrases: human-readable key backup

Bitcoin Improvement Proposal 39 (BIP-39) defines the seed phrase standard: a randomly generated sequence of 12 or 24 words from a fixed 2048-word dictionary encodes the master private key of an HD (Hierarchical Deterministic) wallet. From this single seed, a standard algorithm derives an effectively unlimited number of private keys for different addresses. Backing up the 12 words on paper in a secure location allows complete wallet recovery on any compatible software. The security comes from the size of the word space: 12 words from 2048 possibilities gives $2048^{12} = 2^{132}$ combinations, which is computationally infeasible to brute-force.

Example — Ethereum's account model vs. Bitcoin's UTXO model

Bitcoin tracks value through Unspent Transaction Outputs (UTXOs): every coin is a specific output of a past transaction, and spending requires producing a signature over that specific UTXO. Ethereum tracks value through accounts: each address has a balance, and a transaction simply says "deduct from A, add to B." Bitcoin's UTXO model provides stronger privacy (each transaction creates new outputs at new addresses) but is more complex for smart contracts. Ethereum's account model is simpler for programming but reveals more about a user's history. Both require the same underlying digital signature mechanism — only the data being signed differs.

6 The Role of Randomness in Blockchain Security

Cryptographic security in blockchain depends critically on high-quality randomness for key generation and signature nonces.

Example — The blockchain-specific randomness problem

A blockchain node generating a new wallet key pair must produce a private key that is truly random and unique across all past and future keys ever generated. If two users accidentally generate the same private key — a 256-bit integer chosen uniformly at random from 2^{256} possibilities — both users can spend each other's coins. The probability of this collision is so astronomically small ($\approx 2^{-256}$ per pair generated) that it is considered impossible in practice. However, weak random number generators (predictable seeds, low-entropy sources) have caused real wallet thefts, as in the Android ECDSA vulnerability discussed earlier.

? Think about it

Why is generating randomness so hard in practice, even on modern computers?

✓ Answer

Computers are fundamentally deterministic machines: given the same starting state, they produce the same output. True randomness must come from a physical source — hardware entropy from CPU timing jitter, keyboard timing, hard drive seek times, or dedicated hardware random number generators. Operating system random number generators (/dev/urandom on Linux, CryptGenRandom on Windows) collect this physical entropy and mix it with a cryptographic function to produce pseudo-random output that is computationally indistinguishable from true randomness. Problems arise on embedded devices (no keyboard, minimal I/O), at system startup (low accumulated entropy), or when virtual machines are cloned (identical starting state).

7 Commitment Schemes in Blockchain

Definition — Commitment Scheme

A commitment scheme allows a party to "commit" to a value without revealing it, and later "open" the commitment to prove the original value. It requires two properties: **hiding** (the commitment reveals nothing about the committed value before opening) and **binding** (the committer cannot change the committed value after making the commitment). Formally, $\text{Commit}(m; r) = c$ (using randomness r) and $\text{Open}(c, m, r) = \text{true/false}$.

Example — The Pedersen commitment in confidential transactions

The Pedersen commitment $C = r \cdot G + v \cdot H$ (where G, H are elliptic curve base points, v is the transaction value, and r is a blinding factor) hides the value v behind a point on the elliptic curve. Critically, Pedersen commitments are **additively homomorphic**: $C_1 + C_2 = (r_1 + r_2) \cdot G + (v_1 + v_2) \cdot H$. This means a network node can verify that transaction outputs sum to the same total as inputs (no money created or destroyed) by checking commitment arithmetic — without ever learning the individual amounts. This is the core of Confidential Transactions, proposed by Greg Maxwell and deployed in Monero and several sidechains.

? Think about it

Give me an example of that from the real world.

✓ Answer

Monero, the privacy-focused cryptocurrency, uses Pedersen commitments for all transaction amounts and ring signatures for sender privacy. No observer — including law enforcement or competing users — can determine the amount sent in any Monero transaction. This design has been both praised (financial privacy for legitimate users) and criticized (potential use in illegal transactions), illustrating the real-world tension between privacy and regulatory transparency.

8 Zero-Knowledge Proofs: A Brief Preview

Zero-knowledge proofs (ZKPs) were studied in depth in the IoT Security module (Session 4). In blockchain, they serve as the key technology for scaling (Layer 2 ZK-Rollups) and privacy (Zcash's

zk-SNARKs).

Example — ZK-Rollups: processing thousands of transactions off-chain

An Ethereum ZK-Rollup (such as zkSync or StarkNet) processes 10,000 transactions off-chain on a specialized server called a "prover." The prover generates a zero-knowledge proof that all 10,000 transactions are valid (signatures check out, no double-spends, correct balance updates) without submitting all 10,000 transactions to Ethereum. Only the proof (a few hundred bytes) is submitted on-chain. Ethereum's smart contract verifies the proof in a single transaction. This achieves 100–1000x higher throughput than processing each transaction individually on-chain.

Example — Zcash: private transactions using zk-SNARKs

Zcash introduced shielded transactions using zk-SNARKs (Succinct Non-Interactive Arguments of Knowledge) in 2016. A sender constructs a proof that: (1) they know the spending key for the input notes; (2) the total input value equals the total output value; (3) each output note is correctly formed. The verifier (any network node) checks the proof in milliseconds without learning who sent what to whom, or for how much.

? Think about it

Why are ZK-Rollups considered one of the most promising scaling solutions for public blockchains?

✓ Answer

Because they inherit Ethereum's full security without requiring users to trust the ZK-Rollup operator. Even if the prover (the centralized sequencer running the rollup) is malicious and produces invalid transactions, the zero-knowledge proof will fail, and Ethereum's smart contract will reject the batch. Users can always exit the rollup by submitting a proof of their balance directly to Ethereum, meaning the operator cannot steal funds. This "trust-minimized" design is what distinguishes ZK-Rollups from earlier "optimistic" rollups that rely on a fraud-proof challenge period.

9 Cryptographic Comparison: IoT vs. Blockchain Contexts

The same primitives serve different specific purposes in the two contexts:

Primitive	IoT role (earlier module)	Blockchain role (this module)
Hash functions	Data integrity, firmware verification, certificate fingerprints	Block linking, Merkle tree, PoW puzzle
Digital signatures	Device authentication, firmware signing	Transaction authorization, Taproot multi-sig
ECDH / asymmetric	TLS handshake, key exchange	Off-chain data encryption, encrypted messaging
Homomorphic encryption	Privacy-preserving IoT analytics	Confidential smart contracts, private auctions
Zero-knowledge proofs	Device attestation, range proofs	ZK-Rollups (scaling), Zcash (privacy)
Commitment schemes	Session 7 verification tools	Pedersen commitments, confidential transactions

Example — A complete worked example: signing and broadcasting a Bitcoin transaction

Step 1 — Construct the transaction: "Input: UTXO (0.5 BTC owned by Alice). Output: Pay 0.49 BTC to Bob's address, 0.01 BTC fee to miner." Step 2 — Hash: $h = \text{SHA256}(\text{SHA256}(\text{tx}))$. Step 3 — Sign using Alice's private key sk: compute ECDSA signature $\sigma = \text{Sign}(sk, h)$. Step 4 — Broadcast (tx, σ) to the Bitcoin network. Step 5 — Any node verifies: extract Alice's public key pk from the UTXO's locking script, check $\text{Verify}(pk, h, \sigma) = 1$, confirm the UTXO is unspent, confirm amounts balance. Step 6 — If all checks pass, the transaction waits in the mempool until a miner includes it in a block. This entire process takes milliseconds and requires no central authority.

Session Summary

This session reviewed cryptographic primitives specifically in their blockchain roles. Hash functions provide block linking, Merkle tree transaction integrity, and the Proof of Work mining puzzle, exploiting the avalanche effect and pre-image resistance. ECDSA provides transaction authentication; Schnorr signatures (Taproot) add privacy and multi-signature aggregation. Asymmetric encryption appears in off-chain data encryption and encrypted peer-to-peer messaging. Homomorphic encryption enables confidential smart contracts. Commitment schemes (Pedersen) underpin confidential transactions by allowing balance verification without revealing amounts. Zero-knowledge proofs scale blockchains (ZK-Rollups) and add privacy (Zcash zk-SNARKs). The comparison table shows how the same primitives serve different specific purposes in IoT versus blockchain contexts. Session 4 covers the remaining blockchain-specific cryptographic mechanism: consensus algorithms.

? Think about it

Looking at the comparison table, what property appears both in IoT contexts and blockchain contexts but is used for fundamentally different goals in each?

✓ Answer

Zero-knowledge proofs. In IoT (IoT Security module, Session 4), they serve device attestation and privacy-preserving range proofs — proving a device's identity or that a sensor reading

falls within a valid range, without disclosing the raw value. In blockchain, the same tool serves scaling (allowing thousands of transactions to be verified off-chain with a single proof) and full transaction privacy (hiding sender, receiver, and amount from all observers). The underlying mathematics is identical; the application context determines which aspect of the proof's properties is most exploited.